



MANIPAL
UNIVERSITY JAIPUR
University under Section 2(f) of the UGC Act

Centre for Distance & Online Education

Manipal University Jaipur

Internal Assignment Submission – April 2025

Programme: Bachelor of Computer Applications (BCA)

Semester: IInd

Course Code & Name: DCA1208 -DBMS

Assignment Sets: Set-1 & Set-2

Session: April 2025

Submitted By:

Name: Hirdyansh Varshney

Roll Number: 2414500172

Email ID: 2414500172@muonline.edu.in

Declaration:

I hereby declare that the content of this assignment is original and has been prepared by me.

It has not been copied from any source, and plagiarism has been strictly avoided.

Signature: Hirdyansh Varshney

Set -1

Question : 1 (a)

Answer : A Database System is a software system that allows users to create, manage, and manipulate data efficiently. Based on their structure, data handling, and architecture, database systems can be classified into the following types:

1. Centralized Database System:

- All data is stored at a single location.
- Users access it remotely through a network.
- Suitable for organizations with simple data needs.

2. Distributed Database System:

- Data is distributed across multiple physical locations.
- Ensures better reliability and speed.
- Often used in banks or multinational companies.

3. Relational Database System (RDBMS):

- Stores data in tables (relations).
- Supports SQL for querying.
- Examples include MySQL, PostgreSQL, and Oracle.

4. Object-Oriented Database System:

- Stores data as objects (used in OOP).
- Useful in applications requiring complex data relationships.

5. Hierarchical and Network DBMS:

- Hierarchical: Data organized in a tree structure (parent-child).
- Network: Allows many-to-many relationships between data.

Conclusion:

The choice of a database system depends on the complexity, scalability, and structure of data in any organization or application.

(b)

Answer : Entity Integrity and Referential Integrity are two important rules in relational databases that ensure data consistency and correctness.

Entity Integrity:

- Ensures that each record in a table is uniquely identified.
- Applied using the Primary Key, which must be unique and not null.

- Example: Student_ID in the Students table must have a unique value for every student.

Referential Integrity:

- Maintains consistency between related tables.
- Enforced using Foreign Keys, which must refer to an existing value in another table.
- Example: Course_ID in the Enrollments table must match a valid Course_ID in the Courses table.

Key Differences:

Entity Integrity	Referential Integrity
Based on Primary Key	Based on Foreign Key
Ensures uniqueness in a table	Ensures valid references between tables
Null values not allowed	Null values allowed

Conclusion:

While entity integrity focuses on individual records, referential integrity ensures that relationships between tables are valid. Both are critical for maintaining a reliable relational database.

Question 2 :

Answer : In the context of relational databases, keys are special fields or combinations of fields used to identify and maintain relationships between records in tables. They ensure uniqueness, integrity, and help in retrieving data efficiently.

What is a Key?

A key is a constraint used to uniquely identify rows in a table. Without keys, there would be duplicate or inconsistent records, which would reduce the reliability of a database.

Types of Keys:

1. Primary Key:

- A field (or group of fields) that uniquely identifies each record.
- Cannot contain NULL values.
- Each table can have only one primary key.

Example:

In a Students table, Student_ID can be a primary key.

2. Candidate Key:

- All possible fields that can uniquely identify a record.
- One of them becomes the primary key.
- All candidate keys are unique and not null.

Example:

Both Student_ID and Email_ID can be candidate keys in the same table.

3. Super Key:

- A **superset of candidate keys**.
- May include additional attributes that are not necessary for uniqueness.

Example:

{Student_ID, Name} is a super key, but not a candidate key.

4. Foreign Key:

- A field in one table that refers to the primary key of another table.
- Used to establish relationships between tables.

Example:

Course_ID in the Enrollments table referencing the Course_ID in Courses table.

5. Alternate Key:

- Candidate keys that were **not chosen as the primary key**.
- Can still uniquely identify records.

Example:

If Student_ID is the primary key, then Email_ID becomes an alternate key.

6. Composite Key:

- A key formed by combining two or more columns to uniquely identify a record.
- Used when no single field can uniquely identify records.

Example:

In a Result table, combination of Student_ID and Subject_Code forms a composite key.

Example Table:

Student_ID	Email_ID	Course_ID
101	abc@gmail.com	BCA101
102	xyz@gmail.com	BCA102

- Student_ID → Primary Key
- Email_ID → Alternate Key
- Course_ID → Foreign Key (referencing Courses table)

Conclusion:

Keys are essential in a relational database to ensure data accuracy, uniqueness, and maintain relationships between tables. Understanding different types of keys allows for efficient database design and supports data integrity throughout the system.

Question 3 :

Answer : The Entity-Relationship (ER) Model is a high-level conceptual data model used to define the structure of a database in a simple and visual way. It was introduced by Peter Chen in 1976 and is widely used in the design phase of a database before implementing it physically.

The ER model represents real-world entities and their relationships in the form of ER diagrams, using symbols to show how data is connected.

Definition:

An ER Model defines the entities, their attributes, and the relationships between them. It is helpful for visualizing the logical structure of a database in a way that's easy to understand and design.

Components of ER Diagram:

1. Entity:

- An entity is an object or thing in the real world that is distinguishable from other objects.
- Represented by a rectangle in ER diagrams.
- Example: Student, Course, Teacher.

2. Entity Set:

- A collection of similar types of entities.
- Example: All students in a university.

3. Attributes:

- Properties or characteristics of an entity.
- Represented by ellipses.
- Example: Name, Age, Student_ID.

Types of attributes:

- Simple Attribute – Cannot be divided (e.g., Age)
- Composite Attribute – Can be divided (e.g., Name → First Name + Last Name)
- Derived Attribute – Can be derived from other values (e.g., Age from DOB)
- Multivalued Attribute – Can have multiple values (e.g., Phone numbers)

4. Relationship:

- A relationship is an association among two or more entities.
- Represented by a diamond shape.
- Example: A Student enrolls in a Course.

5. Relationship Set:

- A set of relationships of the same type.
- Example: All "enroll" relationships between students and courses.

6. Cardinality (Mapping Constraints):

- Specifies the number of entities that can be associated with another entity.
- Types:
 - One-to-One (1:1)
 - One-to-Many (1:N)
 - Many-to-One (N:1)
 - Many-to-Many (M:N)

7. Keys:

- An attribute (or combination) that uniquely identifies an entity.
- Represented with an underlined ellipse in diagrams.

Example ER Diagram Components:

Let's say we have two entities: Student and Course

- Attributes of Student: Student_ID, Name
- Attributes of Course: Course_ID, Title
- Relationship: Enrolled (Student → Course)

Conclusion:

The ER model is an essential tool in database design. It allows developers and analysts to model the data in terms of entities, attributes, and relationships, and helps in translating real-world situations into structured database designs. Components like entities, attributes, relationships, and mapping constraints form the core of any ER diagram.

Set – 2

Question 4 :

Answer : In database design, functional dependency is a concept that forms the foundation for the process of normalization. It helps to remove redundancy and ensure data integrity in relational databases.

What is Functional Dependency?

A functional dependency (FD) is a relationship between two attributes (or sets of attributes) in a relation.

Formally, $X \rightarrow Y$ means:

"If two tuples (rows) have the same value for attribute X, they must also have the same value for attribute Y."

- X is the determinant
- Y is the dependent

Example:

In a Student table:

If $\text{Student_ID} \rightarrow \text{Student_Name}$, it means that for a given Student_ID, the name must be uniquely determined.

Importance of Functional Dependencies:

- Helps to identify key attributes in a table.
- Plays a vital role in detecting data anomalies.
- Used to normalize database tables by splitting them logically.

What is Normalization?

Normalization is the process of organizing data in a database to reduce data redundancy and improve data integrity. It is done using functional dependencies and normal forms (1NF, 2NF, 3NF, etc.).

How Functional Dependencies Influence Normalization:

Functional dependencies are the basis for every normal form.

1. First Normal Form (1NF):

- Ensures that all attributes are atomic (no repeating groups).
- FDs help identify repeating values.

2. Second Normal Form (2NF):

- Based on full functional dependency.
- A table is in 2NF if it is in 1NF and every non-prime attribute is fully dependent on the entire primary key.

3. Third Normal Form (3NF):

- Based on transitive dependencies.
- A table is in 3NF if it is in 2NF and non-prime attributes are only dependent on the primary key, not on other non-prime attributes.

Example:

Consider a table:

Student_ID Name Course_ID Course_Name

101 Aditi BCA101 DBMS

102 Rahul BCA101 DBMS

Functional Dependencies:

- Student_ID $\rightarrow$ Name
- Course_ID $\rightarrow$ Course_Name

Here, Course_Name is dependent on Course_ID, not on Student_ID, so this is a transitive dependency.

To remove this, we split the table into two:

1. Student(Student_ID, Name, Course_ID)
2. Course(Course_ID, Course_Name)

This is how normalization uses functional dependencies to reduce repetition and anomalies.

Conclusion:

Functional dependencies help in understanding how attributes relate to each other and play a crucial role in designing normalized database schemas. By analyzing FDs, we can remove redundancy, prevent anomalies, and create efficient, consistent databases.

Question 5 :

Answer : Let's assume the Student table has the following structure:

Student(Student_ID, Name, Course_ID, Marks)

a. Find the name of students whose name ends with 'n':

SELECT Name

FROM Student

WHERE Name LIKE '%n';

Explanation:

The % symbol is a wildcard. '%n' means the name can start with anything but must end with 'n'.

b. Display records of students who belong to BCA103:

SELECT *

FROM Student

WHERE Course_ID = 'BCA103';

Explanation:

This retrieves all columns (*) for students whose Course_ID matches 'BCA103'.

c. Display student records in increasing order of marks:

```
SELECT *  
FROM Student  
ORDER BY Marks ASC;
```

Explanation:

ORDER BY Marks ASC sorts the results in ascending (increasing) order of marks.

d. Change the student's name from Diksha to Deepak:

```
UPDATE Student  
SET Name = 'Deepak'  
WHERE Name = 'Diksha';
```

Explanation:

This command updates the name only where the current name is 'Diksha'.

e. Delete the record of the student whose ID is 107:

```
DELETE FROM Student  
WHERE Student_ID = 107;
```

Explanation:

This removes the record for the student having Student_ID 107.

Conclusion:

These SQL queries demonstrate how to retrieve, update, sort, and delete records in a student table using simple SQL commands. Writing clear and correct SQL is essential for managing real-world databases efficiently.

Question 6(a) :

Answer : In DBMS, a transaction is a single logical unit of work that consists of one or more database operations like INSERT, DELETE, UPDATE, or SELECT. A transaction is treated as a single indivisible operation, meaning it must be either fully completed or fully failed — no in-between.

Example:

If a bank transfers ₹1000 from Account A to Account B, both debit from A and credit to B must succeed. If one fails, the entire transaction must be rolled back.

Transaction States:

1. Active:

- Initial state when a transaction starts execution.

2. Partially Committed:

- All operations are completed, but changes are not yet permanent.

3. **Committed:**

- All operations are successfully executed, and changes are permanently saved.

4. **Failed:**

- Some error or failure occurs before completion.

5. **Aborted:**

- Transaction is rolled back due to failure, and the database is restored to its original state.

Conclusion:

Transaction states help the DBMS to track and manage transactions effectively, ensuring data remains accurate and consistent even in case of failure or system crash.

(b)

Answer : A good database transaction must follow the ACID properties, which are essential to ensure data accuracy, consistency, and reliability.

ACID Properties of a Transaction:

1. Atomicity:

- A transaction is all-or-nothing.
- If any part of the transaction fails, the entire transaction fails, and all changes are rolled back.

2. Consistency:

- A transaction must take the database from one valid state to another.
- It must follow all integrity constraints (like primary key, foreign key rules).

3. Isolation:

- Transactions should be executed independently.
- Intermediate results of one transaction must be invisible to others.

4. Durability:

- Once a transaction is committed, its changes must be permanent, even in case of a system crash.

Example:

In a bank system, transferring money involves multiple operations. Using ACID properties ensures:

- No partial transfer (Atomicity)
- Balances remain correct (Consistency)

- Multiple transfers don't interfere (Isolation)
- Transaction is safely saved (Durability)

Conclusion:

The ACID properties ensure that transactions are safe, reliable, and recoverable, which is essential for maintaining trust and consistency in any database system.